Assignment 06

Published | Assign To | Edit | ⋮

Learning Objectives

The objective of this assignment is to learn how to auto-configure our web application with details for the database server and set up autorun for our service so that it starts up without requiring any manual intervention from the administrator.

Infrastructure as Code w/Terraform

IAM Roles & Policies Updates

Warning

- The assignment objective requires you to identify the permissions needed and create appropriate roles and policies.
- Not all required roles and policies are listed in this assignment or on the lecture page.

S3 Bucket

- Create a private S3 bucket with a bucket name that is a UUID.
- Make sure Terraform can delete the bucket even if it is not empty. Before you delete the bucket, you can use the following aws cli command to delete all objects from it manually using CLI, which may work. `aws s3 rm s3://bucket-name --recursive`.
- Enable **default encryption for S3 Buckets (https://docs.aws.amazon.com/AmazonS3/latest/dev/bucket-encryption.html)**.
- Create a lifecycle policy for the bucket to transition objects from `STANDARD` storage class to `STANDARD_IA` storage class after `30 days`.

Create DB Security Group

- Create an `EC2 security group` for your `RDS` instances.
- Add `ingress` rule to allow TCP traffic on the port `3306` for `MySQL/MariaDB` or `5432` for `PostgreSQL`.
- The `source` for the traffic should be the `application` security group.
- Restrict access to the instance from the internet.
- This security group will be referred to as the `database` security group.

RDS Parameter Group

A DB parameter group acts as a container for engine configuration values applied to one or more DB instances. Create a new parameter group to match your database (Postgres, MySQL, or MariaDB) and its version. Then, the RDS instance must use the new parameter group, not the default one.

RDS Instance

Setting Public accessibility to True will expose your instance to the internet.

Your RDS instance should be created with the following configuration. You may use default values/settings for any property not mentioned below.

Property	Value
Database Engine	MySQL/MariaDB/PostgreSQL
DB Instance Class	cheapest one
Multi-AZ deployment	No
DB instance identifier	csye6225
Master username	csye6225
Master password	pick a strong password
Subnet group	**Use the private subnet for RDS instances**
Public accessibility	No
Database name	csye6225

The Database security group should be attached to this RDS instance.

User Data

- The EC2 instance should be launched with user data.

- Database configurations such as username, password, and hostname should be passed on to the web application using user data.

Web Application

- The web application's database must be the RDS instance launched with Terraform when running on the EC2 instance.
- You can no longer install or use the local database on the EC2 instance. However, you will use the local database for your integration tests in your GitHub Actions.

Web Application API Requirements

You can find the API specification here **https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a6 (https://app.swaggerhub.com/apis-docs/csye6225-webapp/cloud-native-webapp/fall2025-a6)**

1. All API request/response payloads should be in JSON.
2. No UI should be implemented for the application.
3. As a user, I expect all API calls to return with a proper **HTTP status code (https://en.wikipedia.org/wiki/List_of_HTTP_status_codes)** .
4. Users should be authenticated & authorized where applicable.
5. Users can upload images to the products they have created.
    1. You must add support for popular file types such as jpeg, jpg, png, etc.
    2. Each image must be stored in an S3 bucket.
    3. Object metadata must be stored in the database. See AWS docs for available **metadata (https://docs.aws.amazon.com/AmazonS3/latest/dev/UsingMetadata.html#object-metadata)** .
    4. Users can (hard) delete images they have uploaded.
        1. The image must be deleted from the S3 bucket.
    5. Users cannot update their images.
6. Users can delete only their own images from the products they have created.
7. Users should not be able to delete images uploaded by other users or from products created by other users.
8. Multiple users can upload images with the same name. You must ensure the partitioning user's images in the object storage bucket.
9. S3 credentials should not be hardcoded anywhere, and the application must be able to access S3 using the IAM role attached to the EC2 instance.

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories

- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Grading Guidelines

Warning: The following guidelines are for information only and are subject to change at the discretion of the instructor and TA.

Common Grading Rubrics

Git

1. All commits to GitHub repositories must be made via pull request. No changes can be made by pushing commits directly to the main branch. The exception to this is the first commit made to the repository to initialize it, which can be done directly from the UI.
2. All pull requests must have status checks and be enforced. Merging pull requests that did not pass status checks may result in penalties.

Security

1. Committing any kind of credentials to the code or uploading them as part of your Canvas submission may result in a penalty.
2. Service providers may automatically disable your keys or suspend your account if they detect keys being committed in your GitHub repositories, even if they are private. This can severely affect your assignment grade and your ability to work on assignments in the course. This is not grounds for assignment extensions; late penalties will apply as documented in the syllabus.

Software Engineering Best Practices

1. Values for variables should not be hardcoded in the code. It's OK to set default values for variables, but if the value cannot be modified during the demo or when the TA or instructor asks, you may be penalized.
2. Follow industry standards and use a style guide for your code. You can find them with simple web searches.

Code Stability, Error Handling, & Edge Cases

1. The application should not crash between various API calls.
2. The application, VM, or other resources should not require a restart during the demo unless the grading guidelines require it to demonstrate a specific feature/functionality.
3. The server cannot respond with a 500 Internal Server Error. This usually highlights areas of code that do not have proper error handling.

Previous Assignment Objectives

TAs must verify that students have resolved all open issues from the previous assignment(s).

Git & GitHub Repository

Git Repository Content Check

1. Check the repository for any IDE-specific files. IDE configuration files must not be in the repository.
2. Verify their .gitignore configuration.
3. Check the repository for build artifacts such as `.class`, `.jar`, `.war` files and `build`, `node_modules`, `venv` directory. None of these should be checked into the repository.
4. Check for dependencies. Dependencies from the Maven repository, npm, or Python should not be committed to the git repository.

Git Forking Workflow

1. No direct commits are made to the organization repository. Verify that the student works from a forked repository and uses the feature branch. There should be no direct commits to the main branch in their forked repository.

Infrastructure as Code w/Terraform & Web application

Database security group

1. Verify that the database security group is of the type `EC2 security group`.
2. Verify that `source` of the traffic is the `application security group`. The source cannot be IP ranges.

RDS Parameter Group

1. Verify that a new RDS parameter group is created using Terraform.

RDS Instance

1. Verify that the RDS instance is deployed in `private` subnet created using Terraform code in the VPC created by Terraform code.
2. Verify that the RDS instance is not accessible over the internet.
3. RDS instance must have the custom RDS parameter group attached to it.

EC2 w/User Data

1. Verify that the database hostname is passed to the application using the User Data script.
2. Test API endpoints with POST requests so the database is populated.
3. Stop the RDS instance and test the health check endpoint.
4. SSH to the EC2 instance and check if RDBMS (MySQL, PostgreSQL, MariaDB) is installed locally.
5. Connect to the RDS instance from EC2 (RDBMS cli tools can be installed manually at this point) and query tables to show data (look at users and other tables for data).

SystemD AutoRun

1. Verify the application is online when the EC2 instance is launched.
2. Reboot the EC2 instance and verify that the application is running by calling APIs (POST request to create a new assignment and then GET). Students should not need to `SSH`.
3. SystemD unit set up in the AMI.

Removal of Local RDBMS Install

1. Verify that the Packer template no longer installs the database locally.

Security

1. The application directory and binary are not owned by `admin` user & group.
2. A dedicated non-privileged user is created for the application.
3. SystemD unit file launches the application with the dedicated non-privileged user.
4. File and directory permissions are not set to `777`.

S3 Bucket

- The S3 bucket to store attachments is created using Terraform.
- The S3 bucket should be private.
- The S3 bucket should be encrypted.
- The S3 bucket has lifecycle policies set up to transition all objects from `STANDARD` storage class to `standard_IA` storage class after 30 days.

File Upload

- When deleting an image, the record should be deleted from the database (hard delete), and the file should be deleted from the S3 bucket.

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Oct 23 at 3pm	Everyone	Oct 16 at 6pm	Oct 24 at 3pm

+ Rubric